Homework 1: Face Detection

111550170 卓建均

Part I. Implementation:

Part 1. Load and prepare your dataset

Part 1-1. Preparing data_small dataset

I initialize an empty dataset structured into two lists, one each for training and testing data. Then I navigate through the dataset directory, categorizing images into subsets ('train' and 'test') and labels ('face' and 'non-face'). Using OpenCV to read each image in grayscale mode. Next, I assign labels—faces as 1 and non-faces as 0 which enable the model to learn from the input images. Finally, I append (image, label) pairs to the respective subsets to construct the dataset.

HW1 > dataset.py > load_data_FDDB

```
7   def load_data_small():
17          Returns:
18              dataset: The first and second element represents the training and testing dataset respectively
19      """
20      # Begin your code (Part 1-1)
21      dataset = [], []
22      data_path = 'C:\\Users\\choan\\Desktop\\intro_ai\\1\\HW1\\data\\data_small'
23      subsets = ['train', 'test']
24      for subset in subsets:
25          labels = ['face', 'non-face']
26          for label in labels:
27              label_path = os.path.join(data_path, subset, label) #os.path.join拼接文件路徑
28              label_val = 1 if label == 'face' else 0
29              for root, dirs, files in os.walk(label_path): #os.walk可以遍歷目錄樹
30                  for file in files:
31                      img_path = os.path.join(root, file)
32                      img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE) #imread可以把圖片轉為numpy array
33                      dataset[subsets.index(subset)].append((img, label_val))
34      #raise NotImplementedError("To be implemented")
35      # End your code (Part 1-1)
36      return dataset
```

Part 1. 1-2 Expanding the data_FDDB dataset by generating non-face regions

I iterate through each detected face bounding box and generate non-face regions with these face regions. I first calculate the width and height of each face region. Then randomly select a specified number of non-face regions within each face region. For each non-face region, a random top-left point is chosen within the image dimensions, and the bottom-right point is calculated based on the width and height of the face region. Next, I check for intersection between the non-face region and each face bounding box to avoid overlap with face regions. If no intersection is found, the non-face region is considered valid and cropped from the grayscale image. This non-face region is then resized to a fixed size of 19x19 pixels and added to the non-face dataset with label 0. Finally, I set a break to check only one non-face region per face region.

```
HW1 > dataset.py > load_data_FDDB
39  def load_data_FDDB(data_idx="01"):
97          # Begin your code (Part 1-2)
98          for face_box in face_box_list:
99              left_top_face, right_bottom_face = face_box
100             face_width = right_bottom_face[0] - left_top_face[0]
101             face_height = right_bottom_face[1] - left_top_face[1]
102             # randomly select non-face region
103             for _ in range(num_faces):
104                 #randomly select top-left point for non-face region
105                 nonface_left_top_x = np.random.randint(0, img_gray.shape[1] - face_width)
106                 nonface_left_top_y = np.random.randint(0, img_gray.shape[0] - face_height)
107                 nonface_left_top = (nonface_left_top_x, nonface_left_top_y)
108                 #calculate bottom-right point for non-face region
109                 nonface_right_bottom_x = nonface_left_top_x + face_width
110                 nonface_right_bottom_y = nonface_left_top_y + face_height
111                 nonface_right_bottom = (nonface_right_bottom_x, nonface_right_bottom_y)
112                 #check for intersection with face region
113                 intersection = False
114                 for face_box in face_box_list:
115                     left_top, right_bottom = face_box
116                     if nonface_right_bottom[0] > left_top[0] and nonface_left_top[0] < right_bottom[0] \
117                             and nonface_right_bottom[1] > left_top[1] and nonface_left_top[1] < right_bottom[1]:
118                         intersection = True
119                         break
120                 if not intersection:
121                     nonface_img_crop = img_gray[nonface_left_top[1]:nonface_right_bottom[1],
122                                                 nonface_left_top[0]:nonface_right_bottom[0]].copy()
123                     nonface_dataset.append((cv2.resize(nonface_img_crop, (19, 19)), 0))
124                     break #break to only select one non-face region per face region
125         #for i in range(num_faces):
126         #raise NotImplementedError("To be implemented")
127         # End your code (Part 1-2)
```

Part 2. Implement Adaboost algorithm

I iterate through each feature and polarity combination to find the weak classifier with the lowest classification error. I initialize best error as infinity and best classifier as None. For each feature and polarity, I create a weak classifier and calculate its classification error. If the current error is lower than the previous best error, it updates the best error and best classifier accordingly. Finally, it returns the best classifier and its corresponding error.

```
HW1 > adaboost.py > Adaboost > selectBest
 11  class Adaboost:
146      def selectBest(self, featureVals, iis, labels, features, weights):
158              The ith element is the weight assigned to the ith training sample.
159          Returns:
160            bestClf: The best WeakClassifier Class
161            bestError: The error of the best classifer
162          """
163          # Begin your code (Part 2)
164          bestError = np.inf
165          bestClf = None
166          for feature in features:
167              for polarity in [1, -1]:
168                  clf = WeakClassifier(feature, polarity)
169                  predict = np.array([clf.classify(ii) for ii in iis])
170                  error = np.sum(weights * (predict != labels))
171                  if error < bestError:
172                      bestError = error
173                      bestClf = clf
174          # End your code (Part 2)
175          return bestClf, bestError
```

Part 4. Detect face

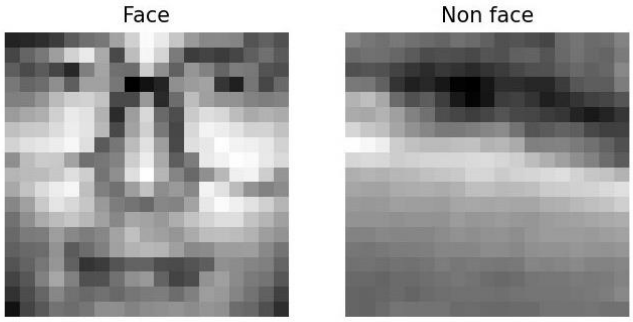
First, it reads data from a (.txt) file in the detect folder which contains image names and the number of faces in each image. For each image, it reads the image, converts it to grayscale, and iterates through the detected faces. Using a trained weak classifier, it classifies each detected face as either a face or a non-face. Bounding boxes are drawn around the detected faces, with green indicating a classified face and red indicating a non-face. The results are displayed using matplotlib.

```
HW1 > detection.py > detect
  9  def detect(dataPath, clf):
 21      # Begin your code (Part 4)
 22      folderPath = 'C:\\Users\\choan\\Desktop\\intro_ai\\1\\HW1\\data\\detect'
 23      with open(dataPath, 'r') as file:
 24          lines = file.readlines()
 25          line_index = 0
 26          while line_index < len(lines):
 27              parts = lines[line_index].strip().split()
 28              if len(parts) != 2:
 29                  line_index += 1
 30                  continue
 31              image_name, num_faces = parts
 32              num_faces = int(num_faces)
 33              img_path = os.path.join(folderPath, image_name)
 34              img = cv2.imread(img_path)
 35              gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
 36              line_index += 1
 37              for _ in range(num_faces):
 38                  parts = lines[line_index].strip().split()
 39                  if len(parts) != 4:
 40                      line_index += 1
 41                      continue
 42                  x, y, w, h = map(int, parts)
 43                  face_img = gray_img[y:y+h, x:x+w]
 44                  face_resized = cv2.resize(face_img, (19, 19))
 45                  if clf.classify(face_resized):
 46                      color = (0, 255, 0)
 47                  else:
 48                      color = (0, 0, 255)
 49                  cv2.rectangle(img, (x, y), (x+w, y+h), color, thickness = 2)
 50                  line_index += 1
 51              plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
 52              plt.axis('off')
 53              plt.show()
 54      #raise NotImplementedError("To be implemented")
 55      # End your code (Part 4)
```

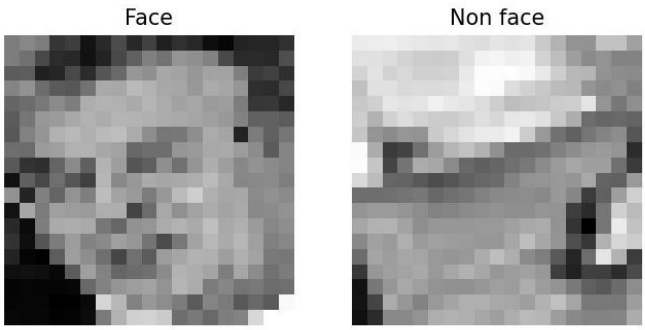
Part II. Results & Analysis:

Part 1. Load and prepare your dataset

The result of first and last image of training dataset when loading data_small



The result of first and last image of training dataset when loading data_FDDB



Part 2. Implement Adaboost algorithm

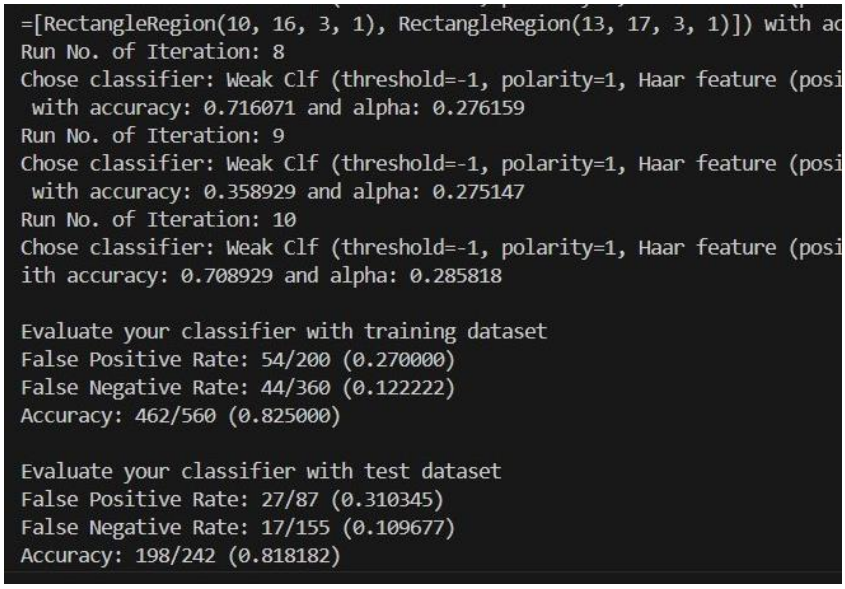
The result after running selectBest function in adaboost.py when loading data_small and T=10

```
Chose classifier: Weak Clf (threshold=-1, polarity=1, Haar feature (positi
ith accuracy: 0.390000 and alpha: 0.889997
Run No. of Iteration: 9
Chose classifier: Weak Clf (threshold=-1, polarity=1, Haar feature (positi
ectangleRegion(2, 9, 2, 2), RectangleRegion(4, 11, 2, 2)]) with accuracy:
Run No. of Iteration: 10
Chose classifier: Weak Clf (threshold=-1, polarity=1, Haar feature (positi
]) with accuracy: 0.345000 and alpha: 0.654148

Evaluate your classifier with training dataset
False Positive Rate: 14/100 (0.140000)
False Negative Rate: 2/100 (0.020000)
Accuracy: 184/200 (0.920000)

Evaluate your classifier with test dataset
False Positive Rate: 30/100 (0.300000)
False Negative Rate: 52/100 (0.520000)
Accuracy: 118/200 (0.590000)
```

The result after running selectBest function in adaboost.py when loading data_FDDB and T=10



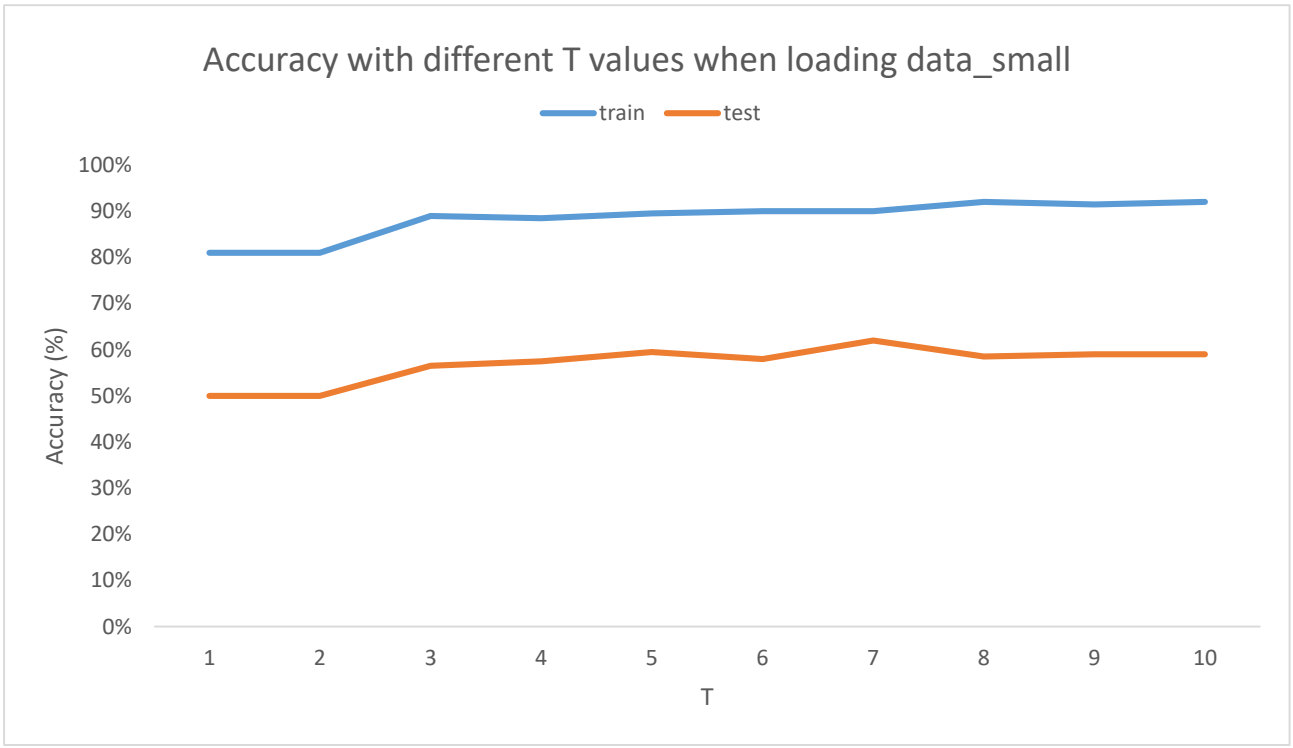
Part 3. Additional experiments

The result of different accuracy by loading different dataset and testing T between 1-10

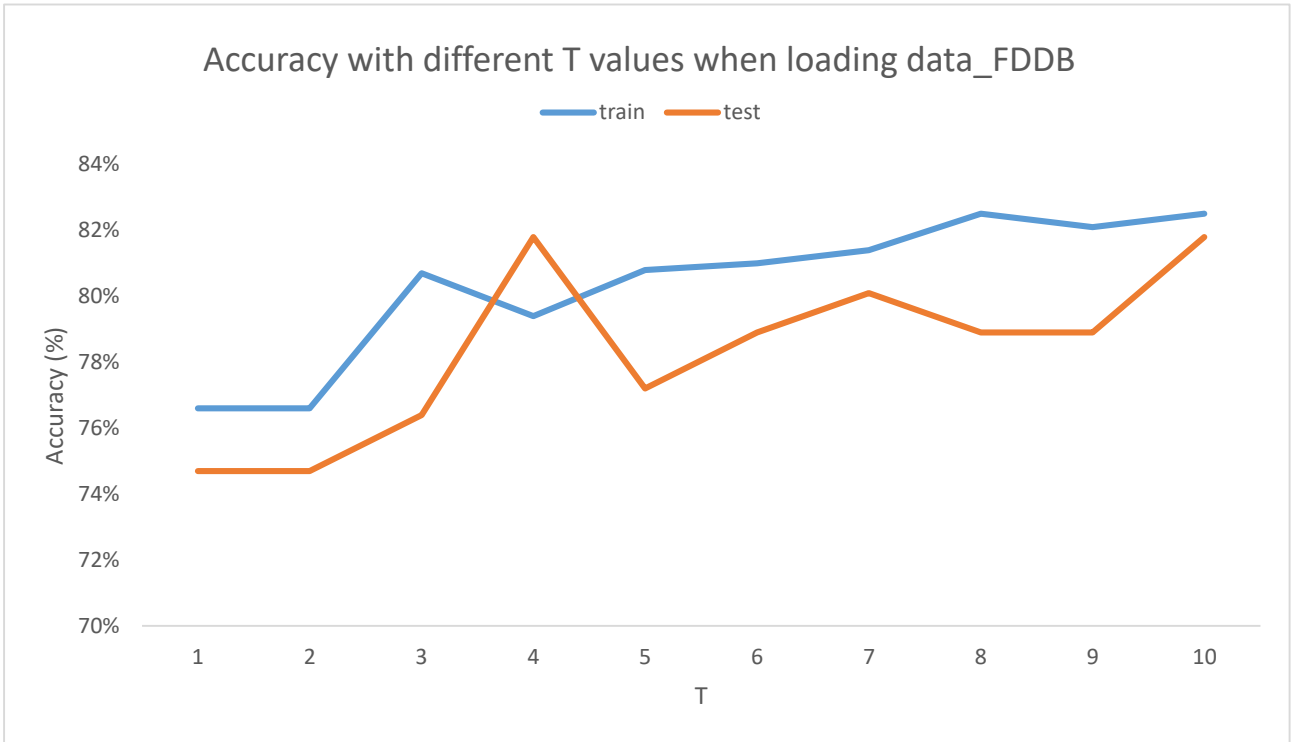
	data_small		data_FDDB	
	training accuracy(%)	testing accuracy(%)	training accuracy(%)	testing accuracy(%)
T = 1	81.0%	50.0%	76.6%	74.7%
T = 2	81.0%	50.0%	76.6%	74.7%
T = 3	89.0%	56.5%	80.7%	76.4%
T = 4	88.5%	57.5%	79.4%	81.8%
T = 5	89.5%	59.5%	80.8%	77.2%
T = 6	90.0%	58.0%	81.0%	78.9%
T = 7	90.0%	62.0%	81.4%	80.1%
T = 8	92.0%	58.5%	82.5%	78.9%
T = 9	91.5%	59.0%	82.1%	78.9%
T = 10	92.0%	59.0%	82.5%	81.8%

According to the results, it's observed that the accuracy tends to improve with larger values of T in the boosting process. However, there appears to be a saturation point beyond which further increases in T do not consistently lead to higher accuracy. When comparing results obtained from different datasets, the model trained on data_FDDB generally outperforms the one trained on data_small. Additionally, the evaluation of the classifier with the test dataset demonstrates higher accuracy compared to evaluations with the training dataset in both cases. Furthermore, it's notable that the accuracy peaks at T=10 for data_FDDB and T=7 for data_small.

The line chart of accuracy with different T values when loading data_small dataset



The line chart of accuracy with different T values when loading data_FDDB dataset



Part 4. Detect face

The face detection result for "the-beatles.jpg" with T = 10 using the data_small dataset.



The face detection result for " p110912sh-0083.jpg" with T = 10 using the data_small dataset.



The face detection result for "the-beatles.jpg" with T = 10 using the data_FDDB dataset.



The face detection result for " p110912sh-0083.jpg" with T = 10 using the data_FDDB dataset.



Part 5. Test classifier on my own images

The face detection result for my own image with T = 10 using the data_small dataset.



The face detection result for my own image with T = 10 using the data_FDDB dataset.



Part 6. Implement another classifier

This approach I compute the error rates for various thresholds and select the parameters that minimize error first, then creates weak classifiers based on these parameters to compute weighted errors for each. It first iterates through features, sorts them based on values, and calculates errors for all thresholds. The best feature, threshold, and polarity are selected for each weak classifier. Then, it classifies samples, computes weighted errors, and selects the best classifier.

```
146     def selectBest(self, featureVals, iis, labels, features, weights):
177         #Part 6 start
178         t_pos, t_neg = 0, 0
179         clfs = []
180         for i in range(len(weights)):
181             if labels[i] == 1:
182                 t_pos += weights[i]
183             else:
184                 t_neg += weights[i]
185         for idx, feature in enumerate(featureVals):
186             applied = sorted(zip(weights, feature, labels), key=lambda x: x[1])
187             pos_cnt, pos_weights, neg_cnt, neg_weights = 0, 0, 0, 0
188             error, best_f, best_t, best_p = float('inf'), None, None, None
189             for w, f, label in applied:
190                 e = min(neg_weights + t_pos - pos_weights, pos_weights + t_neg - neg_weights)
191                 if e < error:
192                     error = e
193                     best_f = features[idx]
194                     best_t = f
195                     if pos_cnt > neg_cnt:
196                         best_p = 1
197                     else:
198                         best_p = -1
```

```
199                 if label == 1:
200                     pos_cnt += 1
201                     pos_weights += w
202                 else:
203                     neg_cnt += 1
204                     neg_weights += w
205             clfs.append(WeakClassifier(best_f, best_t, best_p))
206         bestClf, bestError = None, float('inf')
207         for clf in clfs:
208             eps = 0
209             for i in range(len(iis)):
210                 if clf.classify(iis[i]) != labels[i]:
211                     eps += weights[i]
212             if eps < bestError:
213                 bestClf = clf
214                 bestError = eps
215         return bestClf, bestError
216         #Part 6 end
```

The accuracy using this approach by loading different dataset and testing T between 1-10

	data_small		data_FDDB	
	training accuracy(%)	testing accuracy(%)	training accuracy(%)	testing accuracy(%)
T = 1	94.0%	68.0%	78.5%	71.9%
T = 2	94.0%	68.0%	78.5%	71.9%
T = 3	100.0%	62.5%	84.2%	80.5%
T = 4	100.0%	62.5%	85.1%	81.4%
T = 5	100.0%	71.0%	88.7%	83.8%
T = 6	100.0%	63.5%	88.2%	83.8%
T = 7	100.0%	72.5%	91.6%	85.5%
T = 8	100.0%	74.0%	89.8%	83.8%
T = 9	100.0%	73.0%	91.0%	83.8%
T = 10	100.0%	64.5%	90.5%	83.8%

It's obvious that the accuracy is higher than the original method in both using data_small and data_FDDB.

The result of detecting my image using this approach when T = 7



Part III. Answer the questions:

1. Please describe a problem you encountered and how you solved it.

A: First, the challenge I encountered primarily revolved around managing the data types of various variables throughout the algorithm's execution because I'm not familiar with python really much. I spend a lot of time reading the code, understanding what the inputs and outputs of each function are. Moreover, I read many articles about Adaboost and Viola-Jones' algorithm to understand each component well and the main steps described in the code. The second challenge I encountered was obtaining the coordinates of faces in my own image. To address this, I write a code to help me to extract the coordinates of each face within my image. This was accomplished after conducting research online to find a suitable method to accomplish the task.

```
import cv2
# use cascadeclassifier
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')
#read the image
img = cv2.imread('C:\\Users\\choan\\Desktop\\intro_ai\\1\\HW1\\data\\detect\\p110912sh-0083.jpg')
#turn the picture into gray scale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
#face detect
faces = face_cascade.detectMultiScale(gray, 1.3, 5)
#open file to save face coordinates
with open('C:\\Users\\choan\\Desktop\\intro_ai\\1\\HW1\\data\\detect\\testing.txt', 'w') as file:
    #write the picture name and the number of detected faces
    file.write(f'{len(faces)}\n')
    for (x, y, w, h) in faces:
        #write the coordinates of each face
        file.write(f'{x} {y} {w} {h}\n')
```

I use the OpenCV library for face detection. Firstly, it loads a pre-trained face recognizer model using the CascadeClassifier class. Then, it reads an image and converts it to grayscale for face detection. Next, it detects faces in the grayscale image using the face recognizer. Finally, it saves the detected face coordinates to a .txt file I built.

2. How do you generate "nonface" data by cropping images?

A: First, I iterate through each face box in the face_box_list to calculate the width and height of the face region. Then randomly select a top-left point for a non-face region within the image boundaries and calculate the bottom-right point of the non-face region by adding the face width and height to the respective coordinates. Next, I check if there is an intersection between the non-face region and the detected face regions. If yes, it discards the current non-face region and randomly selects a new one. Once a non-face region without intersection is found, it crops the corresponding area from the grayscale image and resizes it to a fixed size of 19x19 pixels. This cropped non-face image, along with a label 0 is then added to the non-face dataset.

3. What are the limitations of the Viola-Jones' algorithm?

A: There are some limitations of the Viola-Jones algorithm. First, it operates with fixed-size windows for feature extraction, potentially limiting scalability across different object sizes. Moreover, it's sensitive to orientation, necessitating separate classifiers for various orientations, which escalates model complexity and training overheads. It also struggles with illumination

variations and complex backgrounds. Additionally, it may yield false positives and negatives, relying on the quality and diversity of training data.

4. **Based on Viola-Jones' algorithm, how to improve the accuracy except changing the training dataset and parameter T?**

A: To enhance the accuracy of the Viola-Jones algorithm, a promising approach involves exploring alternative feature sets beyond the traditional Haar-like features. This means exploring more advanced ways to extract features or using deep learning to let the algorithm learn features directly from the data. By adopting features that capture a broader range of object characteristics and variations, the algorithm can improve its ability to discriminate between objects and backgrounds, leading to enhanced detection accuracy, especially in challenging scenarios.

5. **Other than Viola-Jones' algorithm, please propose another possible face detection method (no matter how good or bad, please come up with an idea). Please discuss the pros and cons of the idea you proposed, compared to the Adaboost algorithm.**

A: One approach I propose involves detecting the color composition and positioning within an image to discern human faces. For instance, human eyes typically feature a combination of white and black, while lips tend to be red, and eyebrows black. By analyzing these color patterns and their spatial arrangement, we can identify potential faces. The pros is that it can detect faces at various angles since human faces' color remains consistent, unlike traditional methods reliant on specific orientations. However, the cons is that if it's in low-light conditions where facial features might appear monochromatic, making detection challenging and likely to fail.